

Part 1 — Common Foundation (Learn Once, Use Everywhere)

This foundation is genuinely non-negotiable across all four domains. These are the skills that every technical interview will test, regardless of the specific job title.

1. Programming & Python

- **What to learn:** Python syntax, control flow, functions, object-oriented programming (OOP), error handling, and working with virtual environments.
- **Important Subtopics:** List comprehensions, decorators, generators, file I/O, and basic unit testing (`pytest`).
- **Depth Required:** You must be able to write clean, modular Python code to solve data manipulation and automation problems. You should be comfortable debugging and reading documentation.
- **Why Common:** Python is the *lingua franca* for data engineering (Airflow, Spark), data science (Pandas, Scikit-learn), and AI/ML (PyTorch, Hugging Face).
- **Practical Exercises:**
 1. Build a command-line script that renames and organizes files in a messy directory.
 2. Write a class-based program to manage a simple inventory system, with methods for adding, removing, and searching items.

2. Data Structures & Algorithms (DSA)

- **What to learn:** Arrays, linked lists, hash tables (dictionaries), stacks, queues, trees, and graphs. Basic algorithms: sorting (merge, quick), searching (binary), and Big O notation.
- **Important Subtopics:** Time and space complexity analysis, recursion, and basic dynamic programming.
- **Depth Required:** For freshers, you need to solve **easy to medium LeetCode-style problems**. The goal is not competitive programming; it is to demonstrate logical thinking and clean code.
- **Why Common:** All technical interviews include a DSA/coding round. It proves you can think algorithmically.
- **Practical Exercises:**
 1. Solve 50 easy and 20 medium problems on LeetCode, focusing on arrays, strings, and hash maps.
 2. Implement a LRU (Least Recently Used) cache using a dictionary and a doubly linked list.

3. SQL & Databases

- **What to learn:** Relational database concepts (tables, keys, indexes), SQL syntax (SELECT, WHERE, JOINS, GROUP BY, HAVING), and database design (normalization, ER diagrams).
- **Important Subtopics:** Subqueries, Common Table Expressions (CTEs), Window Functions (`ROW_NUMBER` , `RANK` , `LAG/LEAD`), and query optimization basics (understanding indexes).
- **Depth Required:** You must be able to write complex analytical queries without looking up basic syntax. Window functions and CTEs are **essential for all four roles**. You should understand the difference between `WHERE` and `HAVING` .
- **Why Common:** SQL is the universal language for interacting with structured data. Data Engineers build pipelines with it; Data Scientists query data with it; ML/LLM Engineers often need to fetch and join data for training or RAG.
- **Practical Exercises:**

1. Use a sample e-commerce database (like the “Chinook” or “Northwind” dataset) to answer 20 business questions (e.g., “Find the top 5 customers by total sales in the last year”).
2. Write a query that calculates a running total and a 7-day moving average of daily sales.

4. Linux & Command Line

- **What to learn:** Navigating the file system (`cd` , `ls` , `pwd`), file manipulation (`cp` , `mv` , `rm` , `cat` , `grep`), permissions (`chmod`), and process management (`ps` , `kill`).
- **Important Subtopics:** SSH, environment variables, piping (`|`), and package management (`apt` or `yum`).
- **Depth Required:** You need to be comfortable working in a terminal environment. Most production servers and Docker containers run on Linux.
- **Why Common:** You will deploy, debug, and manage data pipelines, ML models, and LLM applications on Linux-based systems.
- **Practical Exercises:**
 1. Use `grep` , `awk` , and `sed` to extract and transform data from a large log file.
 2. Write a bash script to automate a repetitive task, such as backing up a directory and compressing it.

5. Git & GitHub

- **What to learn:** Version control concepts, the Git workflow (`init` , `add` , `commit` , `push` , `pull`), and branching strategies.
- **Important Subtopics:** Pull Requests (PRs), resolving merge conflicts, `.gitignore` , and writing good commit messages.
- **Depth Required:** You must be able to collaborate on a project using Git. Your GitHub profile is your resume for technical skills.
- **Why Common:** Every software project uses Git. Recruiters will look at your GitHub to see your code quality and project history.
- **Practical Exercises:**
 1. Create a repository for a project, use branches for different features, and merge them via a Pull Request.
 2. Contribute to an open-source project by fixing a documentation typo or a small bug.

6. Statistics & Mathematics

- **What to learn:** Descriptive statistics (mean, median, standard deviation), probability distributions (Normal, Binomial), and inferential statistics (hypothesis testing, p-values, confidence intervals).
- **Important Subtopics:** Linear algebra (vectors, matrices, matrix multiplication), calculus (derivatives, partial derivatives, chain rule), and Bayes’ Theorem.
- **Depth Required:** For Data Scientist and AI/ML Engineer roles, you must understand the *intuition* behind these concepts, not just the formulas. For Data Engineer and LLM Engineer roles, a working knowledge is sufficient.
- **Why Common:** Statistics is the language of data. Linear algebra is the language of deep learning.
- **Practical Exercises:**
 1. Given a dataset, calculate the mean, median, and standard deviation, and create a histogram to visualize the distribution.
 2. Implement a simple linear regression using only NumPy and gradient descent to understand the underlying math.

7. Cloud Fundamentals & APIs

- **What to learn:** Core cloud concepts (IaaS, PaaS, SaaS), major services (compute, storage, databases) on one provider (AWS, GCP, or Azure). REST API principles (HTTP methods, status codes, JSON).
- **Important Subtopics:** AWS S3 (storage), EC2 (compute), and IAM (identity). For APIs: making requests with `requests` in Python.
- **Depth Required:** You need to know how to spin up a virtual machine, store data in a cloud storage bucket, and interact with a REST API.
- **Why Common:** The cloud is the modern data center. APIs are how you integrate different services.
- **Practical Exercises:**
 1. Create an AWS S3 bucket, upload a file, and set a bucket policy.
 2. Write a Python script that calls a public API (e.g., a weather API), parses the JSON response, and saves it to a CSV file.

8. Software Engineering Fundamentals

- **What to learn:** Writing modular, readable code; basic testing; debugging techniques; and an understanding of the software development lifecycle.
 - **Important Subtopics:** Design patterns (basic ones like Factory or Singleton), logging, and code documentation.
 - **Depth Required:** You should write code that another engineer can read and understand. This is a major differentiator for freshers.
 - **Why Common:** You are an engineer first. Your ability to write production-quality code is more important than knowing the latest tool.
 - **Practical Exercises:**
 1. Refactor a messy script into multiple functions and files, with a clear separation of concerns.
 2. Add logging to your code to track the flow of execution and debug errors.
-

Part 2 — Data Engineer Roadmap

Foundation from Part 1: Python, SQL, DSA, Linux, Git, Cloud Fundamentals, APIs, Software Engineering.

Data Engineer-Specific Skills:

Area	Core Concepts & Tools	Why It Matters
Advanced SQL	Window functions, CTEs, query optimization, stored procedures.	You will spend most of your time writing complex SQL to transform data.
Data Modeling	Normalization, Star Schema, Snowflake Schema, Slowly Changing Dimensions (SCD).	You design the structure of data warehouses and lakes.
ETL/ELT	Extract, Transform, Load / Extract, Load, Transform concepts. Batch vs. streaming ingestion.	You build the pipelines that move data from source to destination.
Orchestration	Apache Airflow (DAGs, Operators, Sensors).	You schedule, monitor, and manage complex data pipelines.

Big Data Processing	Apache Spark (PySpark), distributed computing concepts.	You process datasets that are too large for a single machine.
Streaming	Apache Kafka (topics, producers, consumers).	You handle real-time data ingestion and processing.
Data Warehouses & Lakes	Snowflake, BigQuery, Redshift, Delta Lake, Parquet file format.	You store and query massive amounts of structured and semi-structured data.
Docker	Containerization, Dockerfiles, Docker Compose.	You package and deploy your pipelines consistently.
Data Quality	Great Expectations, dbt tests, data validation.	You ensure the data you deliver is accurate and reliable.

Advanced Topics (Good-to-Have): Terraform (Infrastructure as Code), Apache Flink (advanced streaming), Data Mesh concepts, Cloud-specific data engineering certifications (AWS Data Engineer Associate).

Job-Ready Project: A cloud-based ETL pipeline that ingests data from an API, stores it in S3, processes it with Spark, loads it into a data warehouse (e.g., Snowflake), and is orchestrated by Airflow.

Part 3 — Data Scientist Roadmap

Foundation from Part 1: Python, SQL, DSA, Statistics, Git, Cloud Fundamentals, Software Engineering.

Data Scientist-Specific Skills:

Area	Core Concepts & Tools	Why It Matters
Exploratory Data Analysis (EDA)	Pandas, NumPy, Matplotlib, Seaborn.	You spend 70% of your time understanding and cleaning data.
Feature Engineering	Creating new features from raw data, handling missing values, encoding categorical variables.	Good features are often more important than complex models.
Machine Learning	Scikit-learn. Supervised learning (Linear/Logistic Regression, Decision Trees, Random Forests), Unsupervised learning (K-Means, PCA).	You build predictive models to answer business questions.
Model Evaluation	Cross-validation, confusion matrix, precision/recall, F1-score, ROC-AUC.	You must know if your model is actually good and why.
Hypothesis Testing & A/B Testing	T-tests, Chi-square tests, p-values, designing and analyzing experiments.	You use statistics to make data-driven decisions and prove causality.
Business Acumen	Framing business problems as data problems, communicating insights to non-technical stakeholders.	The most valuable data scientists can translate data into business impact.

ML Deployment Fundamentals	Pickle/Joblib for model persistence, basic FastAPI/Flask to create an API.	You need to know how models are put into production, even if you don't do it yourself.
-----------------------------------	--	--

Advanced Topics (Good-to-Have): Time series analysis, recommendation systems, NLP with classical methods, advanced statistical modeling.

Job-Ready Project: An end-to-end data science project that starts with a business problem, involves EDA, feature engineering, model training and evaluation, and concludes with a simple deployment (e.g., a Streamlit app or a FastAPI endpoint).

Part 4 — AI/ML Engineer Roadmap

Foundation from Part 1: Python, DSA, Statistics/Math, SQL, Linux, Git, Software Engineering, Cloud Fundamentals.

AI/ML Engineer-Specific Skills:

Area	Core Concepts & Tools	Why It Matters
Deep Learning	PyTorch (or TensorFlow). Neural networks, backpropagation, activation functions, loss functions, optimizers.	You build and train complex models that go beyond classical ML.
Neural Network Architectures	CNNs (Computer Vision), RNNs/LSTMs (Sequences), Transformers (the foundation of modern AI).	You choose the right architecture for the problem (image, text, audio).
Model Training & Tuning	Hyperparameter tuning (Optuna, Ray Tune), transfer learning, regularization (Dropout, Batch Norm).	You need to make models perform well on real-world data.
MLOps & Deployment	Docker , FastAPI , MLflow (experiment tracking), model versioning, CI/CD for ML.	You are responsible for taking a model from a notebook to a production API.
Model Serving & Monitoring	FastAPI for serving, monitoring for data drift and model decay.	Your model must work reliably in production and you must know when it stops working.
Cloud ML Services	AWS SageMaker, GCP Vertex AI, Azure ML.	You leverage managed services to train and deploy models at scale.

Key Difference vs. Data Scientist: A Data Scientist focuses on *analysis, experimentation, and insights*. An AI/ML Engineer focuses on *building, deploying, and maintaining production ML systems*. The AI/ML Engineer is closer to a software engineer.

Advanced Topics (Good-to-Have): Distributed training, model compression (quantization, pruning), GPU optimization.

Job-Ready Project: A deployed image classification or NLP model. The project must include a Dockerfile, a FastAPI endpoint, and a script to monitor for data drift.

Part 5 — LLM Engineer Roadmap

Foundation from Part 1: Python, DSA, Statistics/Math, Git, Linux, APIs, Software Engineering, Cloud Fundamentals. **Plus AI/ML Engineering foundations (Deep Learning, Transformers).**

LLM Engineer-Specific Skills:

Area	Core Concepts & Tools	Why It Matters
Transformer Architecture	Self-attention, multi-head attention, positional encoding, encoder-decoder vs. decoder-only models.	You must understand the architecture you are building on top of.
Tokenization & Embeddings	BPE, WordPiece, SentencePiece. Vector Databases (Pinecone, ChromaDB, FAISS).	You work with text as numbers and need to store and retrieve them efficiently.
Prompt Engineering	Zero-shot, few-shot, chain-of-thought, ReAct prompting.	You design the instructions that guide the LLM to produce desired outputs.
RAG (Retrieval-Augmented Generation)	Chunking strategies, retrieval, re-ranking, context construction.	You build applications that answer questions based on private, up-to-date data.
LLM APIs & Frameworks	OpenAI API, Hugging Face (transformers , peft), LangChain, LlamaIndex .	You use existing foundation models and frameworks to build applications.
Fine-Tuning	LoRA, QLoRA , PEFT techniques.	You adapt a general-purpose LLM to a specific domain or task.
AI Agents & Tool Calling	Function calling, building agents with LangChain/LangGraph, ReAct agents.	You build autonomous systems that can use tools to complete complex tasks.
LLM Evaluation & Observability	RAGAS , custom evaluation harnesses, tracing with LangSmith.	You measure and improve the quality, cost, and latency of your LLM applications.
Deployment & Cost Optimization	FastAPI, Docker , vLLM, quantization, caching.	You deploy your application and make it efficient and affordable to run.

Key Difference vs. AI/ML Engineer: An AI/ML Engineer trains and builds models from scratch. An LLM Engineer builds *applications on top of* existing foundation models. The LLM Engineer is closer to an application developer with deep expertise in language models.

Advanced Topics (Good-to-Have): Training small language models from scratch, advanced agent architectures, LLMOps.

Job-Ready Project: A deployed, full-stack RAG application. It should have a user interface, use a vector database, implement an evaluation harness, and include observability.

Part 6 — Common vs Specialized Skills Matrix

Skill/Chapter	Common	Data Engineer	Data Scientist	AI/ML Engineer	LLM Engineer
Python	C	S	S	S	S
DSA	C	S	S	S	S
SQL	C	S	S	S	S
Linux	C	S	S	S	S
Git/GitHub	C	S	S	S	S
Statistics/Math	C	A	S	S	S
Cloud Fundamentals	C	S	A	S	S
Software Engineering	C	S	A	S	S
Advanced SQL	-	S	A	-	-
Data Modeling	-	S	A	-	-
ETL/ELT	-	S	-	A	A
Spark	-	S	-	A	-
Kafka	-	S	-	A	-
Airflow	-	S	-	-	-
Docker	-	S	A	S	S
EDA	-	A	S	S	S
Feature Engineering	-	A	S	S	A
Machine Learning	-	A	S	S	A
Deep Learning	-	-	A	S	S
PyTorch	-	-	A	S	S
MLOps	-	A	A	S	S
Transformers	-	-	-	S	S
RAG	-	-	-	A	S
Vector Databases	-	-	-	A	S
Prompt Engineering	-	-	-	A	S
Fine-Tuning	-	-	-	S	S
LLM APIs	-	-	-	A	S
AI Agents	-	-	-	A	S

Legend: **C** = Common Foundation, **S** = Specialized (Core), **A** = Advanced/Optional.

Part 7 — Skill Dependency Map

The learning path is strictly sequential. You cannot skip prerequisites.

Common Foundation (Part 1)

****Data Engineering Path****
Advanced SQL Data Modeling ETL/ELT Basics Airflow Spark Kafka Cloud Data Engineering

****Data Science Path****
Statistics/Probability EDA Feature Engineering Machine Learning Model Evaluation A/B Testing

****AI/ML Engineering Path****
Machine Learning Deep Learning (PyTorch) CNNs/RNNs Transformers MLOps Model Serving

****LLM Engineering Path****
Deep Learning Transformers Tokenization/Embeddings LLM APIs Prompt Engineering RAG VLMs

Key Prerequisites Explained:

- **Python NumPy/Pandas ML Deep Learning Transformers LLM Engineering:** This is the core AI progression. You cannot understand transformers without a deep learning foundation.
- **Python SQL Databases Data Modeling ETL Spark Cloud Data Engineering:** This is the core data engineering progression. You cannot build a data pipeline without understanding the data model.
- **Statistics Hypothesis Testing A/B Testing:** A Data Scientist must master experimentation.
- **Docker FastAPI MLOps LLM Deployment:** Deployment skills are built on top of containerization and API development.

Part 8 — Projects (By Domain and Level)

Data Engineer Projects

Level	Project	Problem Statement	Key Technologies	Skills Demonstrated	GitHub Requirements
1 (Beginner)	API to CSV Pipeline	Fetch data from a public API (e.g., weather), clean it with Pandas, and save it as a CSV.	Python, Pandas, Requests	API interaction, data cleaning, file I/O.	Clean code, clear README, requirements.txt.
1 (Beginner)	Log File Analyzer	Parse a large server log file, extract key metrics (e.g.,	Python, Regex, Linux CLI	String parsing, data aggregation, scripting.	A script that can be run on any log file.

		error rates), and output a summary report.			
1 (Beginner)	Database Design	Design and create a normalized database schema for a simple application (e.g., a blog).	SQL, ER Diagram tool	Data modeling, DDL, DML.	SQL scripts for schema and sample data.
2 (Intermediate)	Airflow ETL Pipeline	Build a DAG that extracts data from a database, transforms it, and loads it into a data warehouse.	Airflow, Python, SQL, PostgreSQL	Orchestration, ETL, dependency management.	Docker Compose to run Airflow and Postgres.
2 (Intermediate)	Spark Data Processor	Process a large CSV file (e.g., 10GB) with PySpark, performing aggregations and joins.	Spark, PySpark, S3	Distributed data processing, performance tuning.	Instructions on how to run on a cloud cluster.
2 (Intermediate)	Streaming with Kafka	Create a producer that sends simulated real-time data to a Kafka topic and a consumer that writes it to a database.	Kafka, Python, PostgreSQL	Streaming concepts, producer/consumer patterns.	Docker setup for Kafka and Zookeeper.
3 (Job-Ready)	Cloud Data Warehouse Pipeline	End-to-end pipeline: Ingest from API S3 Spark transformation Load into Snowflake Orchestrate with Airflow.	AWS S3, Spark, Snowflake, Airflow	Full lifecycle, cloud services, data warehousing.	Architecture diagram, detailed README, Terraform (optional).

3 (Job-Ready)	Data Quality Framework	Implement a data quality check system (e.g., with Great Expectations) that validates data at each stage of a pipeline.	Great Expectations, Python, Airflow	Data quality, testing, monitoring.	A pipeline that fails when data quality checks fail.
3 (Job-Ready)	Real-Time Analytics Dashboard	Build a streaming pipeline (Kafka Spark Streaming Elasticsearch) and visualize with Kibana.	Kafka, Spark Streaming, Elasticsearch, Kibana	Real-time processing, end-to-end streaming.	Dockerized stack, clear setup guide.

Data Scientist Projects

Level	Project	Problem Statement	Key Technologies	Skills Demonstrated	GitHub Requirements
1 (Beginner)	Customer Churn Analysis	Perform EDA on a telecom churn dataset to find key drivers of churn.	Pandas, Matplotlib, Seaborn	EDA, data visualization, storytelling.	Jupyter Notebook with clear markdown explanations.
1 (Beginner)	House Price Prediction	Build a linear regression model to predict house prices based on features.	Scikit-learn, Pandas	Regression, train/test split, model evaluation.	Clean notebook, feature importance analysis.
1 (Beginner)	A/B Test Analysis	Analyze the results of a simulated A/B test to determine if a new feature improved conversion.	SciPy, Statsmodels, Pandas	Hypothesis testing, p-values, confidence intervals.	A report explaining the statistical significance.

2 (Intermediate)	Credit Card Fraud Detection	Build a classification model to detect fraudulent transactions in an imbalanced dataset.	Scikit-learn, Imbalanced-learn	Classification, handling imbalanced data, precision/recall.	Model comparison (Logistic Regression vs. Random Forest).
2 (Intermediate)	Customer Segmentation	Use K-Means clustering to segment customers based on purchasing behavior.	Scikit-learn, Pandas, Matplotlib	Unsupervised learning, clustering, PCA for visualization.	A 3D visualization of the clusters.
2 (Intermediate)	Time Series Forecasting	Forecast future sales for a retail store using time series models.	Statsmodels, Prophet, Pandas	Time series analysis, stationarity, forecasting.	A dashboard showing past sales and future predictions.
3 (Job-Ready)	End-to-End ML Pipeline	Build a pipeline for a real-world dataset (e.g., NYC taxi trips) from ingestion to model deployment.	Scikit-learn, FastAPI, Docker	Full ML lifecycle, deployment, API creation.	A FastAPI endpoint that returns predictions.
3 (Job-Ready)	Recommendation System	Build a collaborative filtering recommendation system for movies or products.	Surprise, Scikit-learn, Pandas	Recommendation algorithms, matrix factorization.	A script that generates top-N recommendations for a user.
3 (Job-Ready)	Business A/B Test Design	Design, run, and analyze an A/B test for a business scenario (e.g., a pricing change).	Python, Statsmodels	Experiment design, power analysis, business impact.	A detailed report with recommendations for the business.

AI/ML Engineer Projects

Level	Project	Problem Statement	Key Technologies	Skills Demonstrated	GitHub Requirements
1 (Beginner)	Image Classifier	Train a simple CNN to classify images from the CIFAR-10 dataset.	PyTorch, Torchvision	Deep learning basics, CNN architecture.	A script to train and evaluate the model.

1 (Beginner)	Sentiment Analysis	Build a model to classify movie reviews as positive or negative using a simple RNN.	PyTorch, NLTK	NLP basics, RNNs, text preprocessing.	A notebook with loss and accuracy curves.
1 (Beginner)	Model API	Deploy a pre-trained model (e.g., ResNet) using FastAPI.	FastAPI, PyTorch	Model serving, API basics.	A Dockerfile and a <code>curl</code> example in the README.
2 (Intermediate)	Transfer Learning	Fine-tune a pre-trained model (e.g., EfficientNet) on a custom dataset (e.g., flower classification).	PyTorch, Torchvision	Transfer learning, data augmentation.	Instructions on how to use your own dataset.
2 (Intermediate)	Experiment Tracking	Use MLflow to track experiments (hyperparameters, metrics, artifacts) for a model.	MLflow, Scikit-learn, PyTorch	MLOps, experiment management, reproducibility.	An MLflow UI screenshot in the README.
2 (Intermediate)	CI/CD for ML	Set up a GitHub Action that runs tests and builds a Docker image when you push code.	GitHub Actions, Docker, Pytest	CI/CD, automated testing.	A <code>.github/workflows</code> file and a passing badge.
3 (Job-Ready)	Production ML System	A deployed model with a full MLOps stack: Docker, FastAPI, MLflow, and a monitoring dashboard.	Docker, FastAPI, MLflow, Prometheus, Grafana	End-to-end MLOps, monitoring, deployment.	Docker Compose file, detailed architecture diagram.
3 (Job-Ready)	Distributed Training	Train a model on a large dataset using PyTorch DistributedDataParallel (DDP).	PyTorch DDP, CUDA	Distributed training, GPU utilization.	A script that runs on a multi-GPU machine.
3 (Job-Ready)	Model Optimization	Take a trained model and optimize it for inference using quantization and pruning.	PyTorch, ONNX, TensorRT	Model optimization, performance benchmarking.	A comparison of latency before and after optimization.

LLM Engineer Projects

Level	Project	Problem Statement	Key Technologies	Skills Demonstrated	GitHub Requirements
-------	---------	-------------------	------------------	---------------------	---------------------

1 (Beginner)	Chatbot with OpenAI API	Build a simple chatbot using the OpenAI API with a basic CLI interface.	Python, OpenAI API	API integration, prompt formatting.	A script that maintains conversation history.
1 (Beginner)	Document Summarizer	Use an LLM to summarize long documents (e.g., PDFs or articles).	Python, LangChain, OpenAI	Context window management, summarization.	A script that accepts a file path and outputs a summary.
1 (Beginner)	Prompt Template Library	Create a collection of reusable prompt templates for different tasks (e.g., code generation, email writing).	Python, YAML/JSON	Prompt engineering, software design.	A well-documented library of prompts.
2 (Intermediate)	RAG over a Knowledge Base	Build a RAG system that answers questions based on a set of private documents (e.g., company policies).	LangChain, ChromaDB, OpenAI	RAG, vector databases, chunking.	A script that ingests documents and answers queries.
2 (Intermediate)	Fine-Tune with LoRA	Fine-tune a small open-source LLM (e.g., Mistral-7B) on a custom dataset using LoRA.	Hugging Face, PEFT, PyTorch	Fine-tuning, LoRA, dataset preparation.	A training script and a demo of the fine-tuned model.
2 (Intermediate)	AI Agent with Tool Calling	Build an agent that can use tools (e.g., a calculator, a weather API) to answer complex queries.	LangChain, ReAct, OpenAI Functions	Agent architecture, tool calling.	A script demonstrating the agent using multiple tools.
3 (Job-Ready)	Full-Stack RAG Application	A deployed web application (e.g., Streamlit) with a RAG backend,	FastAPI, Streamlit, Pinecone, RAGAS, LangSmith	Full-stack, deployment, evaluation, observability.	A live demo link, Dockerfile, and evaluation report.

		evaluation, and observability.			
3 (Job-Ready)	Advanced Agent System	Build a multi-agent system that can collaborate to solve a complex task (e.g., a research assistant).	LangGraph, CrewAI, OpenAI	Multi-agent orchestration, state management.	A clear diagram of the agent architecture.
3 (Job-Ready)	LLM Cost & Latency Optimization	Take an existing RAG application and optimize it for cost and latency (e.g., caching, quantization, smaller models).	vLLM, Redis, quantization, benchmarking	Performance optimization, cost management.	A benchmark comparing before and after optimization.

Part 9 — Job-Ready Roadmap (Per Domain)

Data Engineer

- **Must-Have:** Advanced SQL, Python, Airflow, Spark, a Cloud Platform (AWS/GCP/Azure), Docker.
- **Good-to-Have:** Kafka, dbt, Snowflake/BigQuery/Redshift, Terraform.
- **Optional/Advanced:** Flink, Data Mesh, Kubernetes.
- **Can Skip Initially:** Deep learning, advanced statistics, LLMs.
- **Interview Topics:** SQL (joins, window functions, optimization), Python (data structures, scripting), ETL concepts, Airflow DAGs, Spark basics, cloud services.
- **DSA:** Easy to medium LeetCode.
- **SQL:** Must be excellent. Expect complex queries.
- **Project Requirements:** At least one end-to-end pipeline project with cloud deployment.
- **Resume:** Focus on “built,” “orchestrated,” “processed,” and “scaled.” Quantify data volumes.
- **GitHub:** Show clean, modular code with Dockerfiles and clear READMEs.

Data Scientist

- **Must-Have:** Python (Pandas, Scikit-learn), SQL, Statistics, Machine Learning, EDA.
- **Good-to-Have:** A/B Testing, Time Series, basic deployment (FastAPI, Streamlit).
- **Optional/Advanced:** Deep Learning, Bayesian Statistics, Causal Inference.
- **Can Skip Initially:** Big data tools (Spark), MLOps at scale, LLMs.
- **Interview Topics:** Statistics (p-values, hypothesis testing), ML algorithms (bias-variance, regularization), SQL, case studies, business sense.
- **DSA:** Easy to medium.
- **SQL:** Very important. Window functions and joins are a must.
- **Project Requirements:** A portfolio of 3-4 projects that show the full data science lifecycle. At least one should be deployed.

- **Resume:** Focus on “analyzed,” “predicted,” “optimized,” and “communicated.” Show business impact.
- **GitHub:** Well-documented Jupyter Notebooks that tell a story.

AI/ML Engineer

- **Must-Have:** Python, PyTorch, Deep Learning, MLOps (Docker, FastAPI), SQL.
- **Good-to-Have:** MLflow, Cloud ML services (SageMaker/Vertex AI), CI/CD.
- **Optional/Advanced:** Distributed training, model optimization, Kubernetes.
- **Can Skip Initially:** Advanced statistics, classical ML algorithms (can learn on the job), LLM-specific skills.
- **Interview Topics:** ML fundamentals, deep learning concepts (backprop, loss functions), coding (Python, PyTorch), system design for ML, MLOps.
- **DSA:** Medium LeetCode.
- **SQL:** Moderate. You need to be able to query data for training.
- **Project Requirements:** At least one project that is fully deployed with monitoring.
- **Resume:** Focus on “trained,” “deployed,” “optimized,” and “scaled.” Mention model performance metrics.
- **GitHub:** Code that looks like production software, with tests and Dockerfiles.

LLM Engineer

- **Must-Have:** Python, Deep Learning (Transformers), RAG, Vector Databases, Prompt Engineering, FastAPI.
- **Good-to-Have:** Fine-Tuning (LoRA), LangChain/LlamaIndex, Evaluation frameworks (RAGAS), Docker.
- **Optional/Advanced:** Training models from scratch, advanced agent architectures, LLMOps.
- **Can Skip Initially:** Classical ML, advanced statistics, big data tools.
- **Interview Topics:** Transformer architecture, RAG, prompt engineering, evaluation, coding (Python, API integration), system design for LLM apps.
- **DSA:** Easy to medium.
- **SQL:** Low. You might use it to fetch data for RAG, but it’s not the core focus.
- **Project Requirements:** A deployed RAG or agent application with evaluation and observability.
- **Resume:** Focus on “built,” “deployed,” “evaluated,” and “optimized.” Mention cost/latency improvements.
- **GitHub:** A portfolio of LLM applications, with clear demos and a focus on production-readiness.

Part 10 — Learning Sequence (Consolidated for MCA Fresher)

Phase 1 — Common Foundation (Weeks 1–12)

Focus exclusively on Part 1. Build a strong base in Python, DSA, SQL, Linux, Git, and basic statistics. Do not touch any domain-specific tools yet.

Phase 2 — Exploration (Weeks 13–16)

Spend one week on each domain. Watch a high-level tutorial, build a tiny “Hello World” project, and read job descriptions. The goal is to understand the *differences* and decide which path excites you most.

Phase 3 — Specialization (Months 5–9)

Choose **one** primary path. Dive deep into the specific roadmap (Parts 2–5). Learn the tools, build the projects, and master the concepts. Do not try to learn two specializations simultaneously.

Phase 4 — Projects (Months 7–12)

As you learn, build. By the end of Phase 3, you should have completed at least 3–4 projects (one beginner, one intermediate, one job-ready). Polish your GitHub.

Phase 5 — Job Preparation (Months 10–12)

Start applying. Practice interview questions, refine your resume, and prepare for behavioral rounds. Your projects are your primary talking point.

Part 11 — Time-Based Roadmap

3 Months (Intensive, Common Foundation Only)

- **Month 1:** Python, DSA, Linux, Git.
- **Month 2:** SQL (deep dive), Databases, Statistics.
- **Month 3:** Cloud Fundamentals, APIs, Software Engineering, build 2 small common projects.

6 Months (Foundation + One Specialization)

- **Months 1–3:** Common Foundation (as above).
- **Months 4–6:** Choose one specialization (e.g., Data Engineer). Learn core tools (Airflow, Spark) and build 2 projects.

9 Months (Job-Ready with One Specialization)

- **Months 1–3:** Common Foundation.
- **Months 4–7:** Deep dive into specialization, build 3 projects.
- **Months 8–9:** Advanced topics, interview prep, resume, and applying.

12 Months (Deep Specialization + Portfolio)

- **Months 1–3:** Common Foundation.
 - **Months 4–8:** Deep specialization with advanced topics and MLOps/LLMOps.
 - **Months 9–12:** Build a comprehensive portfolio (4-5 projects), contribute to open source, and apply for jobs.
-

Part 12 — Final Comparison

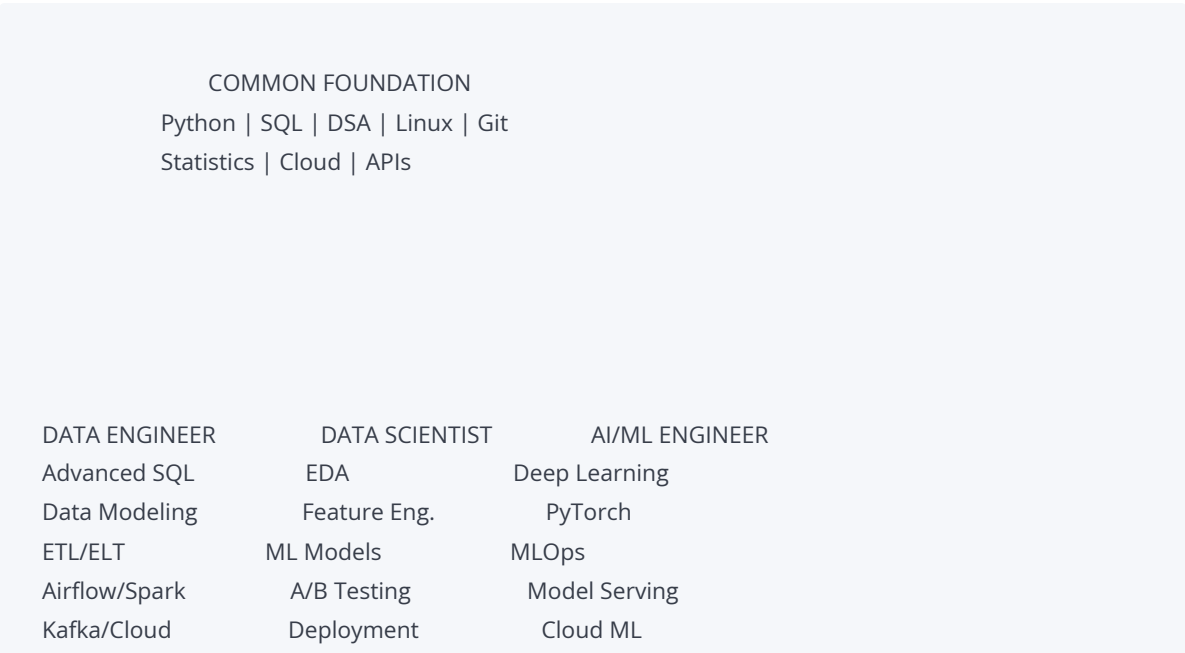
Factor	Data Engineer	Data Scientist	AI/ML Engineer	LLM Engineer
Entry Difficulty	Moderate	High	High	Moderate
Common Foundation Required	Strong	Strong	Strong	Strong
Specialized Knowledge Required	High	High	Very High	High
Mathematics Requirement	Low	High	High	Moderate

Programming Requirement	High	Moderate	Very High	Very High
SQL Requirement	Very High	High	Moderate	Low
ML Requirement	Low	High	Very High	Moderate (DL/Transformers)
Cloud Requirement	High	Low	High	High
Project Complexity	High	Moderate	Very High	High
Typical Fresher Responsibilities	Write SQL, build pipelines, monitor jobs	EDA, build models, create reports	Train/deploy models, write production code	Build RAG apps, fine-tune models, deploy APIs
Main Technologies	SQL, Spark, Airflow, Kafka, Cloud	Python, Pandas, Scikit-learn, SQL	PyTorch, Docker, FastAPI, MLOps	LLMs, LangChain, Vector DBs, FastAPI

Key Role Differences Explained

- **Data Engineer vs. Data Scientist:** The Engineer **builds the roads** (pipelines, infrastructure). The Scientist **drives on the roads** (analyzes data, builds models).
- **Data Scientist vs. AI/ML Engineer:** The Scientist **experiments** to find insights. The Engineer **ships** models to production and ensures they work reliably.
- **AI/ML Engineer vs. LLM Engineer:** The AI/ML Engineer **trains models** (CNNs, RNNs, custom transformers). The LLM Engineer **builds applications** on top of pre-trained foundation models.
- **Data Engineer vs. AI/ML Engineer:** The Data Engineer focuses on **data movement and storage**. The AI/ML Engineer focuses on **model training and deployment**. They work closely together.

Final Visual Hierarchy



LLM ENGINEER
Transformers
RAG
Vector DBs
Fine-Tuning
AI Agents
LLM Deployment